

c0a251964f / ProjExD_2

<> Code

Issues 1

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

main

ProjExD_2 / dodge_bomb.py

Go to file

T

...

c0a251964f 空白追加 #1

a582764 · 2 minutes ago

191 lines (165 loc) · 5.93 KB

Code

Blame

Raw

```
1  import os
2  import sys
3  import pygame as pg
4  import random
5  import time
6
7
8  WIDTH, HEIGHT = 1100, 650
9  os.chdir(os.path.dirname(os.path.abspath(__file__)))
10
11
12  def check_bound(rct: pg.Rect) -> tuple[bool, bool]:
13      """
14      引数: こうかとんRectか、爆弾Rect
15      戻り値: タプル (横方向結果判定, 縦方向結果判定)
16      画面内ならTrue, 画面外ならFalse
17      """
18      yoko, tate = True, True
19      if rct.left < 0 or WIDTH < rct.right:
20          yoko = False
21      if rct.top < 0 or HEIGHT < rct.bottom:
22          tate = False
23      return yoko, tate
24
25
26  def gameover(screen: pg.Surface) -> None:
27      """
28      引数: Screen
29      戻り値: なし
30      ゲームオーバー画面描写
31      """
32      black = pg.Surface((WIDTH, HEIGHT))
33      pg.draw.rect(black, (0, 0, 0), pg.Rect(0, 0, WIDTH, HEIGHT))
34      black.set_alpha(180)
35
36      #文字実装
37      fonto = pg.font.Font(None, 80)
38      txt = fonto.render("Game Over", True, (255, 255, 255))
39      txt_rct = txt.get_rect()
40      txt_rct.center = WIDTH // 2, HEIGHT // 2
41      black.blit(txt, txt_rct)
42
43      # コカトン実装
44      kk_img_l = pg.transform.rotozoom(pg.image.load("fig/8.png"), 0, 0.9)
45      kk_img_r = pg.transform.rotozoom(pg.image.load("fig/8.png"), 0, 0.9)
46      kk_rct_l = kk_img_l.get_rect()
47      kk_rct_r = kk_img_r.get_rect()
48      kk_rct_l.center = WIDTH // 2 - 200, HEIGHT // 2
49      kk_rct_r.center = WIDTH // 2 + 200, HEIGHT // 2
50      black.blit(kk_img_l, kk_rct_l)
51      black.blit(kk_img_r, kk_rct_r)
52
53      #画面描写
54      screen.blit(black, [0, 0])
55      pg.display.update()
56      time.sleep(5)
57
58
59  def init_bb_imgs() -> tuple[list[pg.Surface], list[int]]:
60      """
61      引数: なし
```

https://github.com/c0a251964f/ProjExD_2/blob/main/dodge_bomb.py

```

62     戻り値：タプル（リスト（爆弾の画像），リスト（爆弾の速度））
63     爆弾の大きさと速度を変えるためのタプルを出力
64     """
65     bb_imgs = []
66     for r in range(1, 11): #大きさが異なる爆弾のリストを作成
67         bb_img = pg.Surface((20*r, 20*r))
68         pg.draw.circle(bb_img, (255, 0, 0), (10*r, 10*r), 10*r)
69         bb_img.set_colorkey((0, 0, 0))
70         bb_imgs.append(bb_img)
71     bb_accs = [a for a in range(1, 11)] #爆弾の速度のリストを作成
72     return bb_imgs, bb_accs
73
74
75     def get_kk_imgs() -> dict[tuple[int, int]: pg.Surface]:
76         """
77         引数：なし
78         戻り値：辞書（タプル（x速度, y速度）：向きを変えたものの画像）
79         進む方向ごとにものの向きを変えるための辞書を出力
80         """
81         kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
82         kk_img_flip = pg.transform.flip(kk_img, True, False)
83         kk_dict = {
84             (0, 0): pg.transform.rotozoom(kk_img_flip, 0, 0.9), #何も押していない時
85             (0, -5): pg.transform.rotozoom(kk_img_flip, 90, 0.9), #上
86             (+5, -5): pg.transform.rotozoom(kk_img_flip, 45, 0.9), #右上
87             (+5, 0): pg.transform.rotozoom(kk_img_flip, 0, 0.9), #右
88             (+5, 5): pg.transform.rotozoom(kk_img_flip, -45, 0.9), #右下
89             (0, +5): pg.transform.rotozoom(kk_img_flip, -90, 0.9), #下
90             (-5, -5): pg.transform.rotozoom(kk_img, -45, 0.9), #左下
91             (-5, 0): pg.transform.rotozoom(kk_img, 0, 0.9), #左
92             (-5, +5): pg.transform.rotozoom(kk_img, 45, 0.9), #左上
93         }
94         return kk_dict
95
96
97     def show_life(life_num: int, screen: pg.Surface) -> None:
98         """
99         引数：int(残機), Screen
100        戻り値：なし
101        画面に残機を出力
102        """
103        kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
104        screen.blit(kk_img, [0, 0])
105        fonto = pg.font.Font(None, 80)
106        txt = fonto.render(f" × {life_num}", True, (255, 255, 255))
107        screen.blit(txt, [30, 0])
108
109
110
111     def main():
112         pg.display.set_caption("逃げろ！こうかとん")
113         screen = pg.display.set_mode((WIDTH, HEIGHT))
114
115         #こうかとん初期化
116         bg_img = pg.image.load("fig/pg_bg.jpg")
117         kk_img = pg.transform.rotozoom(pg.image.load("fig/3.png"), 0, 0.9)
118         kk_rct = kk_img.get_rect()
119         kk_rct.center = 300, 200
120
121         #爆弾初期化
122         bb_img = pg.Surface((20, 20))
123         pg.draw.circle(bb_img, (255, 0, 0), (10, 10), 10)
124         bb_img.set_colorkey((0, 0, 0))
125         bb_rct = bb_img.get_rect()
126         bb_rct.center = random.randint(0, 1100), random.randint(0, 650)
127         vx, vy = +5, +5
128
129         clock = pg.time.Clock()
130         tmr = 0
131         DELTA = {
132             pg.K_UP: (0, -5),
133             pg.K_DOWN: (0, +5),
134             pg.K_LEFT: (-5, 0),
135             pg.K_RIGHT: (+5, 0),
136         }
137         bb_imgs, bb_accs = init_bb_imgs()
138         kk_imgs = get_kk_imgs()

```

```
139     life = 3
140     alive = True
141     while True:
142         for event in pg.event.get():
143             if event.type == pg.QUIT:
144                 return
145         if kk_rct.colliderect(bb_rct) and alive:
146             alive = False
147             life -= 1
148             if life == -1:
149                 print("ゲームオーバー")
150                 gameover(screen)
151                 return
152         elif not kk_rct.colliderect(bb_rct):
153             alive = True
154         screen.blit(bg_img, [0, 0])
155
156         show_life(life, screen)
157
158         key_lst = pg.key.get_pressed()
159         sum_mv = [0, 0]
160         for key, mv in DELTA.items():
161             if key_lst[key]:
162                 sum_mv[0] += mv[0] #横方向の移動
163                 sum_mv[1] += mv[1] #縦方向の移動
164         kk_rct.move_ip(sum_mv)
165         if check_bound(kk_rct) != (True, True):
166             kk_rct.move_ip(-sum_mv[0], -sum_mv[1]) #動きをキャンセル
167         kk_img = kk_imgs[tuple(sum_mv)]
168         screen.blit(kk_img, kk_rct)
169
170         avx = vx * bb_accs[min(tmr // 500, 9)]
171         avy = vy * bb_accs[min(tmr // 500, 9)]
172         bb_img = bb_imgs[min(tmr // 500, 9)]
173         bb_rct.width = bb_img.get_rect().width
174         bb_rct.height = bb_img.get_rect().height
175         bb_rct.move_ip([avx, avy])
176         yoko, tate = check_bound(bb_rct)
177         if not yoko:
178             vx *= -1
179         if not tate:
180             vy *= -1
181         screen.blit(bb_img, bb_rct)
182         pg.display.update()
183         tmr += 1
184         clock.tick(50)
185
186
187 if __name__ == "__main__":
188     pg.init()
189     main()
190     pg.quit()
191     sys.exit()
```